

SIMATS ENGINEERING ASSESSMENT

TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS:

Total Marks: 20

1: HTTP Response Analysis

A web server returns the following HTTP response header:

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: -150

Connection: keep-alive

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with Content-Length and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld
    </div>
  </body>
</html>
```

Questions:

- 1. Identify missing or improperly closed elements.
- 2. Analyze how the browser will interpret this structure.
- 3. Determine the impact on rendering and layout.
- 4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

- 1. Interpret the differences in data transmission between GET and POST.
- 2. Analyze which method is more suitable for login and why.
- 3. Identify security risks in using GET for authentication.
- 4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I P Madhavi(912372133) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P Madhavi

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Question 1: Identify any incorrect or invalid fields in the header.

Answer:

The given HTTP response header contains one major error and one minor issue.

The **Content-Length** field is incorrect because it contains a negative value (-150). According to the HTTP protocol, the Content-Length header must always contain a **non-negative integer** that specifies the size of the response body in bytes. A negative value is invalid and violates HTTP standards.

Browsers and clients expect this value to indicate the exact number of bytes they should receive. If the value is incorrect, the response may not be processed correctly.

The **Date** header follows the correct HTTP date format, but the weekday (Mon) should match the actual calendar date. If the weekday does not correspond to the specified date, it creates an inconsistency. Although most browsers ignore this mismatch, it can lead to confusion during debugging, logging, or server synchronization.

The remaining fields are valid:

- **HTTP/1.1** correctly specifies the HTTP protocol version.
- **200 OK** correctly indicates that the server has successfully processed the client's request.
- **Content-Type: text/html** correctly informs the browser that the response contains HTML content.
- **Connection: keep-alive** correctly instructs the browser to keep the TCP connection open for additional requests, reducing the overhead of establishing a new connection.

Therefore, the invalid or incorrect fields are:

- **Content-Length: -150** (Invalid because the value cannot be negative.)
- **Date** (Potential inconsistency if the weekday does not match the actual date.)

Question 2: Analyze the issue with Content-Length and explain its impact.

Answer:

The **Content-Length** header is one of the most important headers in an HTTP response because it tells the browser or client exactly how many bytes of data are contained in the response body. The browser uses this value to determine when the entire response has been received.

In the given response header, the value is:

Content-Length: -150

This is invalid because the HTTP specification requires the Content-Length value to be a **positive integer or zero**. A negative value has no valid meaning and cannot represent the size of any response body.

The impact of this error includes:

- The browser may reject the response because it cannot determine the correct size of the data.
- The webpage may fail to load completely or may display only partial content.
- Some browsers may terminate the connection immediately after detecting the invalid header.
- Network communication errors may occur because the client cannot determine where the response ends.
- Proxy servers, caching systems, and debugging tools may also fail to process the response correctly.

Therefore, the Content-Length header must always contain the exact number of bytes in the response body to ensure reliable communication between the server and the client.

Question 3: Determine whether the status code matches the response correctness.

Answer:

The HTTP status code **200 OK** indicates that the server has successfully received, processed, and fulfilled the client's request. Under normal circumstances, this status code means that the requested resource has been returned successfully without any problems.

However, in this response, although the server returns a **200 OK** status, the response header contains an invalid Content-Length value. This means that while the request itself may have been processed successfully, the response is not fully valid because one of its required headers contains incorrect information.

As a result:

- The server's operation may have been successful.
- The response header is technically incorrect.
- The browser may experience difficulties processing the response.
- Therefore, the **200 OK** status does not completely match the correctness of the response because the response still contains an invalid header field.

A response should only be considered fully correct when both the status code and all header fields comply with the HTTP specification.

Question 4: Suggest corrections for the identified errors.**Answer:**

To make the HTTP response valid, the incorrect header fields should be corrected.

The Content-Length header should contain the actual size of the response body in bytes instead of a negative value. For example, if the HTML document is 150 bytes long, the header should be:

Content-Length: 150

The Date header should also be verified to ensure that the weekday matches the actual calendar date.

The remaining headers can remain unchanged because they already follow the HTTP standard.

Correct HTTP Response Header

HTTP/1.1 200 OK

Date: Wed, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: 150

Connection: keep-alive

Corrections Made:

- Replaced **Content-Length: -150** with **Content-Length: 150**.
- Corrected the weekday in the **Date** header (if necessary).
- Left **HTTP/1.1**, **200 OK**, **Content-Type**, and **Connection** unchanged because they are already valid.

Question 5: Browser–Server Communication Flow**1. Interpret each step in the communication flow**

The communication between a web browser and a web server follows a sequence of steps that enables a user to access a website. Each step has a specific purpose in delivering the requested webpage.

Step 1: User Enters a URL in the Browser

The browser first checks whether the requested webpage is already stored in its cache. If it is not available, the browser prepares to locate the web server that hosts the website.

Step 2: DNS Lookup Occurs

The browser cannot communicate directly using a website name because computers communicate using IP addresses.

Step 3: HTTP Request is Sent

After obtaining the server's IP address, the browser establishes a connection with the server and sends an HTTP or HTTPS request. The request includes information such as the requested webpage, browser type, cookies, headers, and other client details. This request tells the server which resource the user wants to access.

Step 4: Server Processes the Request

The web server receives the HTTP request and processes it. It may retrieve static files such as HTML, CSS, JavaScript, or images, or execute server-side programs to generate dynamic content. If the webpage requires database access, the server queries the database, processes the results, and prepares the appropriate response.

Step 5: Response is Returned

Once the server finishes processing the request, it sends an HTTP response back to the browser. The response contains a status code (such as **200 OK** for success or **404 Not Found** if the page does not exist), response headers, and the requested webpage content. The browser receives this data and prepares it for display.

Step 6: Browser Renders the Page

Finally, the browser parses the HTML document, downloads additional resources such as CSS, JavaScript, fonts, and images, constructs the Document Object Model (DOM), applies styles, executes scripts, and displays the fully rendered webpage to the user. The user can now interact with the website.

2. Identify where delays can occur

Several stages in the browser-server communication process can introduce delays and increase the overall page loading time.

- **DNS Lookup Delay:** If the DNS server takes time to resolve the domain name, the browser must wait before contacting the web server.
- **Network Latency:** Slow internet connections or long physical distances between the client and server can delay data transmission.
- **Server Processing Delay:** The server may take longer to process requests due to heavy traffic, complex application logic, or slow database queries.
- **Database Response Time:** Dynamic websites often depend on databases. Slow database operations can significantly increase response time.
- **Large File Downloads:** Large images, videos, CSS, JavaScript files, and fonts require more time to download.
- **Browser Rendering Delay:** Complex HTML structures, large JavaScript files, or inefficient CSS can slow down rendering.
- **Third-Party Resources:** External advertisements, analytics scripts, social media widgets, and APIs may introduce additional waiting time before the page becomes fully interactive.

These delays directly affect the website's loading speed and user experience.

3. Analyze the role of DNS in the process

The **Domain Name System (DNS)** acts as the Internet's directory service. Humans prefer using domain names such as www.google.com, but computers communicate using numerical IP addresses. DNS bridges this gap by converting readable domain names into machine-readable IP addresses.

The DNS process works as follows:

1. The browser checks its local DNS cache.
2. If not found, the operating system checks its DNS cache.
3. The request is forwarded to the configured DNS resolver.
4. If necessary, the resolver contacts root, top-level domain (TLD), and authoritative DNS servers.
5. The correct IP address is returned to the browser.
6. The browser then connects to the web server using that IP address.

Without DNS, users would need to remember numerical IP addresses instead of simple domain names, making web browsing extremely difficult. Therefore, DNS is an essential component that enables user-friendly access to websites.

4. Suggest optimizations to improve performance

Several optimization techniques can improve browser-server communication and reduce webpage loading time.

- **Enable Browser Caching:** Store frequently accessed resources locally so that repeat visits require fewer network requests.
- **Use DNS Caching:** Cache DNS lookup results to reduce repeated domain name resolution time.
- **Use a Content Delivery Network (CDN):** Serve website content from geographically closer servers, reducing network latency.
- **Compress Files:** Enable Gzip or Brotli compression to reduce the size of HTML, CSS, and JavaScript files before transmission.
- **Minify CSS and JavaScript:** Remove unnecessary spaces, comments, and unused code to decrease download size.
- **Optimize Images:** Compress images and use modern formats such as WebP or AVIF to reduce bandwidth usage.
- **Reduce HTTP Requests:** Combine multiple CSS and JavaScript files where appropriate and eliminate unnecessary resources.
- **Use HTTP/2 or HTTP/3:** These protocols support multiplexing and faster data transfer compared to HTTP/1.1.
- **Optimize Database Queries:** Improve indexing and query efficiency to reduce server processing time.
- **Use Asynchronous Loading:** Load non-critical JavaScript asynchronously or defer its execution so the page can render faster.
- **Implement Lazy Loading:** Load images and videos only when they are about to appear on the user's screen.
- **Keep Connections Alive:** Use persistent connections (Keep-Alive) to avoid repeatedly establishing new TCP connections.

Implementing these optimizations significantly improves website speed, reduces server load, enhances user experience, and improves search engine performance.